

# COMPTE RENDU

Programmation Système — Projet N°3 — Simulation d'une Piscine  
3<sup>e</sup> année Cybersécurité — École Supérieure d'Informatique et du  
Numérique (ESIN)  
Collège d'Ingénierie & d'Architecture (CIA)

**Étudiant :** HATHOUTI Mohammed Taha  
**Filière :** Cybersécurité  
**Année :** 2025/2026  
**Enseignants :** M. Noufel & M. ElAmrani  
**Date :** 22 mars 2026

## Remerciements

Ce projet a été réalisé dans le cadre du cours de Programmation Système dispensé par **M. Noufel** et **M. ElAmrani** à l'UIR durant l'année universitaire 2025-2026.

Je tiens à remercier les enseignants pour la qualité de leur enseignement, et notamment pour les travaux pratiques (TP0 à TP6) qui ont constitué les bases nécessaires à ce projet : gestion des processus, communication inter-processus par tubes, signaux, threads POSIX, mutex et variables de condition sous Linux.

Je souhaite également remercier les enseignants des formations antérieures :

- Les professeurs de l'**Université Grenoble Alpes** (INF301, INF304, INF401) pour la rigueur en programmation et algorithmique ;
- Les professeurs de **Polytech Grenoble** (PROG C, CPS, ALG, ALM) pour la maîtrise du langage C et de la programmation système.

Mohammed Taha HATHOUTI  
Mars 2026

# Table des matières

|          |                                                                                 |          |
|----------|---------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction et Analyse du Problème</b>                                      | <b>3</b> |
| <b>2</b> | <b>Scénario 1 – Avec Risque de Blocage</b>                                      | <b>3</b> |
| 2.1      | Description . . . . .                                                           | 3        |
| 2.2      | Analyse de l'interblocage . . . . .                                             | 3        |
| 2.3      | Implémentation . . . . .                                                        | 3        |
| <b>3</b> | <b>Scénario 2 – Sans Blocage</b>                                                | <b>4</b> |
| 3.1      | Description . . . . .                                                           | 4        |
| 3.2      | Pourquoi ce scénario évite le blocage . . . . .                                 | 4        |
| 3.3      | Implémentation . . . . .                                                        | 4        |
| <b>4</b> | <b>Architecture Modulaire du Programme</b>                                      | <b>5</b> |
| 4.1      | Module <code>semaphores.c</code> – Mémoire partagée et initialisation . . . . . | 5        |
| 4.2      | Module <code>main.c</code> – Création des processus baigneurs . . . . .         | 5        |
| <b>5</b> | <b>Jeux de Tests et Résultats</b>                                               | <b>6</b> |
| 5.1      | Commandes de test . . . . .                                                     | 6        |
| 5.2      | Extrait de résultat – Scénario 2 . . . . .                                      | 6        |
| 5.3      | Analyse des résultats . . . . .                                                 | 6        |
|          | <b>Conclusion</b>                                                               | <b>7</b> |

# 1 Introduction et Analyse du Problème

L'objectif de ce projet est de simuler le fonctionnement d'une piscine en modélisant chaque baigneur comme un **processus concurrent**. Les ressources partagées sont les **paniers** (NP) et les **cabines** (NC, avec  $NC \ll NP$ ).

La contrainte clé est l'ordre d'acquisition des ressources : deux ordres sont étudiés, l'un pouvant conduire à un interblocage (*deadlock*), l'autre en étant exempt.

## Mécanisme de synchronisation choisi : sémaphores POSIX

Les sémaphores POSIX anonymes (`sem_t` avec `pshared=1`) placés dans une zone mémoire partagée obtenue par `mmap(MAP_SHARED | MAP_ANONYMOUS)` sont le choix naturel pour ce projet : ils permettent la synchronisation entre *processus* (et non pas seulement entre threads), ne nécessitent aucun fichier de verrouillage, et offrent une API simple (`sem_wait/sem_post`).

Trois sémaphores sont utilisés :

- **paniers** (compteur, valeur initiale NP) : modélise les paniers disponibles ;
- **cabines** (compteur, valeur initiale NC) : modélise les cabines disponibles ;
- **affichage** (mutex, valeur initiale 1) : protège les `printf` contre l'entrelacement.

## 2 Scénario 1 – Avec Risque de Blocage

### 2.1 Description

Dans ce scénario, le baigneur cherche d'abord **une cabine**, puis **un panier** pour se déshabiller.

#### Ordre d'acquisition – Scénario 1

```
sem_wait(cabines) → sem_wait(paniers) → se déshabille →  
sem_post(cabines) → se baigne → sem_wait(cabines) → sem_post(paniers)  
→ sem_post(cabines)
```

### 2.2 Analyse de l'interblocage

Considérons le cas  $NB = NP = 5$  et  $NC = 3$ . Supposons que les 3 cabines soient occupées par des baigneurs qui attendent chacun un panier (`sem_wait(paniers)`). Si les 5 paniers sont tous détenus par d'autres baigneurs qui, pour les rendre, doivent accéder à une cabine (`sem_wait(cabines)`), **aucun processus ne peut progresser** : c'est un interblocage circulaire.

### 2.3 Implémentation

Listing 1 – `scenario1_nageur()` – ordre cabine puis panier

```
1 static void scenario1_nageur(SharedSems *s, int id) {
```

```

2      /* se deshabiller : cabine d'abord, panier ensuite */
3      sem_wait(&s->cabines);          /* (1) attendre une cabine */
4      sem_wait(&s->paniers);          /* (2) prendre un panier   */
5      sem_post(&s->cabines);          /* libere la cabine       */
6
7      /* se baigner */
8      pause_alea(1000);
9
10     /* se rhabiller */
11     sem_wait(&s->cabines);          /* (3) attendre une cabine */
12     sem_post(&s->paniers);          /* rend le panier         */
13     sem_post(&s->cabines);          /* libere la cabine       */
14 }

```

## 3 Scénario 2 – Sans Blocage

### 3.1 Description

Dans ce scénario, le baigneur prend d'abord **un panier**, *puis* cherche **une cabine**.

#### Ordre d'acquisition – Scénario 2

```

sem_wait(paniers) → sem_wait(cabines) → se déshabille →
sem_post(cabines) → se baigne → sem_wait(cabines) → sem_post(paniers)
→ sem_post(cabines)

```

### 3.2 Pourquoi ce scénario évite le blocage

La condition d'interblocage requiert notamment une *attente circulaire* sur les ressources. En imposant un ordre total d'acquisition ( $\text{panier} < \text{cabine}$ ), aucun cycle d'attente ne peut se former : un baigneur en cabine a **déjà** son panier, il n'en attend pas un autre. Un baigneur en attente de cabine ne bloque aucun panier. L'interblocage est structurellement impossible.

### 3.3 Implémentation

Listing 2 – scenario2\_nageur() – ordre panier puis cabine

```

1 static void scenario2_nageur(SharedSems *s, int id) {
2     /* se deshabiller : panier d'abord, cabine ensuite */
3     sem_wait(&s->paniers);          /* (1) prendre un panier   */
4     sem_wait(&s->cabines);          /* (2) attendre une cabine */
5     sem_post(&s->cabines);          /* libere la cabine       */
6
7     /* se baigner */
8     pause_alea(1000);
9
10    /* se rhabiller */

```

```

11     sem_wait(&s->cabines);           /* (3) attendre une cabine */
12     sem_post(&s->paniers);           /* rend le panier          */
13     sem_post(&s->cabines);           /* libere la cabine        */
14 }

```

## 4 Architecture Modulaire du Programme

Le programme suit une architecture modulaire en trois modules, conformément au style adopté dans les TP précédents :

| Fichier           | Responsabilité                                             |
|-------------------|------------------------------------------------------------|
| semaphores.h / .c | Allocation mmap, init/destroy des sémaphores, log atomique |
| nageur.h / .c     | Logique des deux scénarios, pause_alea()                   |
| main.c            | Parsing argv, fork des NB processus, waitpid               |

### 4.1 Module semaphores.c – Mémoire partagée et initialisation

Listing 3 – semaphores.c – sems\_init()

```

1 SharedSems *sems_init(int NP, int NC) {
2     SharedSems *s = mmap(NULL, sizeof(SharedSems),
3                           PROT_READ | PROT_WRITE,
4                           MAP_SHARED | MAP_ANONYMOUS, -1, 0);
5
6     /* pshared=1 : semaphore partage entre processus distincts */
7     sem_init(&s->paniers, 1, NP);
8     sem_init(&s->cabines, 1, NC);
9     sem_init(&s->affichage, 1, 1); /* mutex affichage */
10    return s;
11 }

```

Le paramètre `pshared=1` de `sem_init` est essentiel : sans lui, les sémaphores ne seraient visibles que dans le processus père et non dans les fils créés par `fork()`.

### 4.2 Module main.c – Création des processus baigneurs

Listing 4 – main.c – fork des NB processus baigneurs

```

1 SharedSems *s = sems_init(NP, NC);
2
3 for (int i = 0; i < NB; i++) {
4     pids[i] = fork();
5     if (pids[i] == 0) {
6         srand(time(NULL) ^ (getpid() << 4)); /* graine unique */
7         if (scenario == 1) scenario1_nageur(s, i);
8         else scenario2_nageur(s, i);
9         exit(EXIT_SUCCESS);
10    }
11 }

```

```

12 for (int i = 0; i < NB; i++)
13     waitpid(pids[i], NULL, 0);
14
15 sems_destroy(s);

```

## 5 Jeux de Tests et Résultats

### 5.1 Commandes de test

Listing 5 – Commandes de compilation et de test

```

1 # Compilation
2 gcc -Wall -Wextra -g -o piscine piscine.c -lpthread
3
4 # Scenario 2 (sans blocage) : 5 baigneurs, 5 paniers, 3 cabines
5 ./piscine 5 5 3 2
6
7 # Scenario 1 (risque blocage) : memes parametres
8 ./piscine 5 5 3 1
9
10 # Scenario 1 avec NB < NP : pas de blocage possible
11 ./piscine 3 5 3 1

```

### 5.2 Extrait de résultat – Scénario 2

Listing 6 – Execution scenario 2 – NB=5 NP=5 NC=3

```

=== Piscine : NB=5 NP=5 NC=3 | Scenario 2 ===

[Baigneur 0] SC2 | prend un panier
[Baigneur 0] SC2 | occupe une cabine pour se deshabiller
[Baigneur 1] SC2 | prend un panier
[Baigneur 2] SC2 | prend un panier
[Baigneur 3] SC2 | prend un panier
[Baigneur 4] SC2 | prend un panier
...
[Baigneur 3] SC2 | libere la cabine -> SORTIE

=== Tous les baigneurs ont quitte la piscine ===

```

### 5.3 Analyse des résultats

| Test        | NB / NP / NC | SC1            | SC2            |
|-------------|--------------|----------------|----------------|
| Cas nominal | 5 / 5 / 3    | Risque blocage | Pas de blocage |
| NB réduit   | 3 / 5 / 3    | Pas de blocage | Pas de blocage |
| NB > NP     | 8 / 5 / 3    | Blocage assuré | Pas de blocage |

Dans le scénario 2, l'ordre d'affichage des baigneurs n'est pas déterministe (ordre d'ordonnement des processus par le noyau), ce qui illustre bien leur exécution réellement concurrente.

## Conclusion

Ce projet a permis de mettre en œuvre les sémaphores POSIX comme mécanisme de synchronisation entre processus pour résoudre un problème classique d'allocation de ressources.

Le point fondamental est que **l'ordre d'acquisition des ressources détermine entièrement la présence ou l'absence de blocage** : le scénario 1 (cabine → panier) crée une attente circulaire possible, tandis que le scénario 2 (panier → cabine) brise ce cycle en imposant un ordre total d'acquisition.

L'utilisation de `mmap(MAP_SHARED)` pour partager les sémaphores entre processus distincts, combinée à `pshared=1` dans `sem_init`, constitue le mécanisme IPC approprié dans un contexte de `fork()` sans fichiers de verrouillage.



## Références

- **Sujet** : M. Noufel & M. ElAmrani, *Programmation Système — Projet N°3 : Simulation d'une Piscine*, UIR ESIN, 2025-2026
- **Cours** : M. Noufel & M. ElAmrani, *TP0 à TP6 — Processus, Tubes, Signaux, Threads et Synchronisation*, UIR ESIN, 2025-2026
- **Cours** : Prof. Eric GASCARD & Prof. Nicolas PALIX, *C Programmation Système (CPS)*, S5, Polytech Grenoble INP, 2024-2025
- **Cours** : Prof. Michael PERIN, *Programmation C (PROG C)*, S5, Polytech Grenoble INP, 2024-2025
- **Problème classique** : E. W. Dijkstra, *Cooperating Sequential Processes*, Technical Report, Eindhoven University of Technology, 1965
- **Ouvrage** : W. Richard Stevens, Stephen A. Rago, *Advanced Programming in the UNIX Environment*, 3<sup>e</sup> édition, Addison-Wesley, 2013
- **Assistant IA** : Claude (Sonnet 4.6), Anthropic, <https://claude.ai>